Reinforcement Learning

Assignment #2: REINFORCE with Baseline Report

Deadline: 2026.04.20

Name: Hangyeol Lee

Student ID: 2021320042

---

1. Introduction

This report describes the implementation of the **REINFORCE with baseline** algorithm applied to the LunarLander-v3 task from OpenAI Gymnasium. The objective is to train an agent that reliably lands between the two flags and achieves an episode reward exceeding 200.

LunarLander-v3 is a continuous-control environment in which an agent controls a lunar lander using a 2-dimensional continuous action space. `action[0]` corresponds to the main (bottom) engine throttle (0.0: off, 1.0: maximum thrust), and `action[1]` controls the side (left/right) engine throttle (-1.0: maximum right, 0.0: off, 1.0: maximum left).

The state space is an 8-dimensional continuous vector consisting of the lander's horizontal and vertical position, horizontal and vertical velocity, tilt angle, angular velocity, and two binary flags indicating whether the left and right legs are in contact with the ground.

An episode terminates (**terminated**) when the lander crashes or moves out of bounds, and is truncated (**truncated**) when the maximum number of steps is reached. The reward is higher when the lander is closer to the target landing pad, moving slowly, and nearly upright. A fuel penalty is applied proportional to main engine usage, a landing bonus is given when both legs touch the ground, and a large negative reward is incurred upon crash.

2. REINFORCE with Baseline Implementation

Based on the provided base code, REINFORCE with baseline algorithm was implemented including the four core components specified in the instructions: Actor-critic policy(2.1 and 2.2), Stochastic action sampling(2.3), Trajectory collection(2.4), Return/advantage estimation(2.5 and 2.6).

2.1. Policy Network

The policy network π_θ determines which action to take in a given state. The network takes the 8-dimensional state vector as input and passes it through two shared hidden layers with ReLU activations. The output is split into two separate linear heads: a **mean head** μ_θ and a **standard deviation head** σ_θ, each producing a 2-dimensional vector corresponding to the two action dimensions. The standard deviation σ_θ is passed through a softplus activation and a small constant (10^{-5}) is added to guarantee a positive lower bound, preventing numerical instability during log-probability computation.

Layer	**Input Dim**	**Output Dim**	**Activation**
Linear (shared)	8	64	ReLU
Linear (shared)	64	64	ReLU
Linear — mean head μ_θ	64	2	—
Linear — std head σ_θ	64	2	Softplus $+10^{-5}$

Table 1: Policy network architecture. The shared backbone feeds into separate mean and std heads.

2.2. Value Network (Baseline)

The value network V_φ serves as the baseline in the REINFORCE algorithm. It predicts the expected cumulative return from a given state, and its output is subtracted from the Monte Carlo return to form the advantage estimate. This reduces the variance of the policy gradient without introducing bias.

The network takes the same 8-dimensional state vector as input. Unlike the policy network, it has a single output head that produces a scalar estimate $V_\varphi(s)$, representing the expected return from state s.

Layer	**Input Dim**	**Output Dim**	**Activation**
Linear	8	64	ReLU
Linear	64	64	ReLU
Linear — value head V_φ	64	1	—

Table 2: Value network architecture. A single scalar output estimates the expected return $V_\varphi(s)$.

2.3. Stochastic Action Sampling

The action selection mode is determined by the boolean parameter `stochastic` in the `select_action` function.

During training (stochastic = True): A Gaussian distribution is constructed using the mean μ_θ and standard deviation σ_θ produced by the policy network. Assuming the two action dimensions are independent, the distribution is expressed as a multivariate Gaussian with a diagonal covariance matrix.

$$u \sim \mathcal{N}\big(\mu_\theta, \mathrm{diag}(\sigma_\theta^2)\big)$$

Since a Gaussian distribution is unbounded, directly using the sampled values may produce actions outside the continuous action range $[-1, 1]$ of LunarLander-v3. To prevent this, tanh is applied to the sample u to squash the action into the $[-1, 1]$ range.

$$a = \tanh(u)$$

Since tanh is a differentiable change of variables rather than a simple clipping operation, the change-of-variables theorem requires adding a Jacobian correction term to the log-probability.

$$\log \pi_\theta(a|s) = \log \mathcal{N}(u; \mu_\theta, \sigma_\theta) - \sum_i \log(1 - \tanh^2(u_i))$$

However, directly computing $\log(1 - \tanh^2(u))$ becomes numerically unstable for large $|u|$, as $\tanh(u) \to \pm 1$. To avoid this, the following softplus-based equivalent expression is used, following the implementation of Achiam (2018).

$$\log \pi_\theta(a|s) = \log \mathcal{N}(u; \mu_\theta, \sigma_\theta) - \sum_i 2(\log 2 - u_i - \text{softplus}(-2u_i))$$

This is derived from the identity $\log(1 - \tanh^2(u)) = 2(\log 2 - u - \text{softplus}(-2u))$. Under the independence assumption, the joint log-probability is computed as the sum over both action dimensions.

During evaluation (stochastic = False): Instead of sampling from the distribution, a deterministic action is obtained by applying tanh directly to the distribution mean.

$$a = \tanh(\mu_\theta)$$

2.4. Trajectory Collection

At the start of each training episode, the environment is reset and actions are sampled and executed consecutively until the episode terminates or is truncated. At each time step t, the following quantities are collected to form the trajectory with length T: the log-probability $\log \pi_\theta(a_t|s_t)$ of the selected action, the reward r_t received from the environment, and the baseline value $V_\varphi(s_t)$ estimated by the value network. The full trajectory is collected before any network update takes place.

2.5. Return and Advantage Estimation

From the collected trajectory, the discounted cumulative return G_t is computed as follows.

$$G_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$$

The advantage A_t is obtained by subtracting the baseline $V_\varphi(s_t)$ from the return, and represents how much better the taken action was compared to the expected value of the current state.

$$A_t = G_t - V_\varphi(s_t)$$

A positive advantage ($A_t > 0$) indicates the action led to a better-than-expected outcome, while a negative advantage ($A_t < 0$) indicates the opposite.

Since G_t is a Monte Carlo estimate over a full episode, the scale of returns can vary significantly across episodes or even within a single episode, potentially destabilizing the policy

gradient. To address this, the advantages are normalized across all time steps of the episode before being used for the policy update.

$$\hat{A}_t = \frac{A_t - \mu_A}{\sigma_A + \varepsilon}$$

where μ_A and σ_A are the mean and standard deviation of all advantages in the episode, and $\varepsilon = 10^{-8}$ prevents division by zero. This normalization keeps the gradient signal at a consistent scale across episodes, improving training stability.

2.6. Policy and Value Loss

Policy loss: The policy network is updated to maximize the expected return weighted by the normalized advantage. Since gradient descent minimizes the loss, the objective is negated as follows.

$$\mathcal{L}_\pi(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log \pi_\theta(a_t|s_t)\hat{A}_t$$

A positive advantage increases the log-probability of the taken action, while a negative advantage decreases it.

Value loss: The value network is updated to minimize the mean squared error between the predicted state value and the Monte Carlo return.

$$\mathcal{L}_V(\varphi) = \frac{1}{T}\sum_{t=1}^{T} \left(V_\varphi(s_t) - G_t\right)^2$$

The two networks are updated independently using separate Adam optimizers, with the value network updated before the policy network each episode.

2.7. Hyperparameters

Parameter	Value	Description
M	3000	Number of training episodes
γ	0.99	Discount factor
η_π	3×10^{-4}	Policy network Adam learning rate
η_V	1×10^{-3}	Value network Adam learning rate

Table 3: Hyperparameters used in training.

3. Training Procedure

The full training loop is summarised below.

1. For each episode $m = 1, ..., M$:
    1. Reset the environment to obtain the initial state s_1.
    2. Collect a trajectory: for each time step t until the episode terminates or is truncated:
        1. Sample a stochastic action $a_t \sim \pi_\theta(\cdot \mid s_t)$ and record $\log \pi_\theta(a_t \mid s_t)$.
        2. Execute a_t; observe reward r_t and next state s_{t+1}.

   3. Record the baseline value $V_\varphi(s_t)$.
   3. Compute the discounted return G_t for each time step via reverse accumulation.
   4. Compute the advantage $A_t = G_t - V_\varphi(s_t)$ and normalize across the episode to obtain $\hat{A}_t$.
   5. Update the value network by minimizing $\mathcal{L}_V$ using Adam.
   6. Update the policy network by minimizing $\mathcal{L}_\pi$ using Adam.
   7. Record episode reward, policy loss, and value loss.
2. After training, evaluate the policy deterministically for 50 episodes and retain the 5 highest-reward videos.
3. Save the performance graph showing episode rewards, policy loss, and value loss over training.

4. Results

4.1. Performance Graph

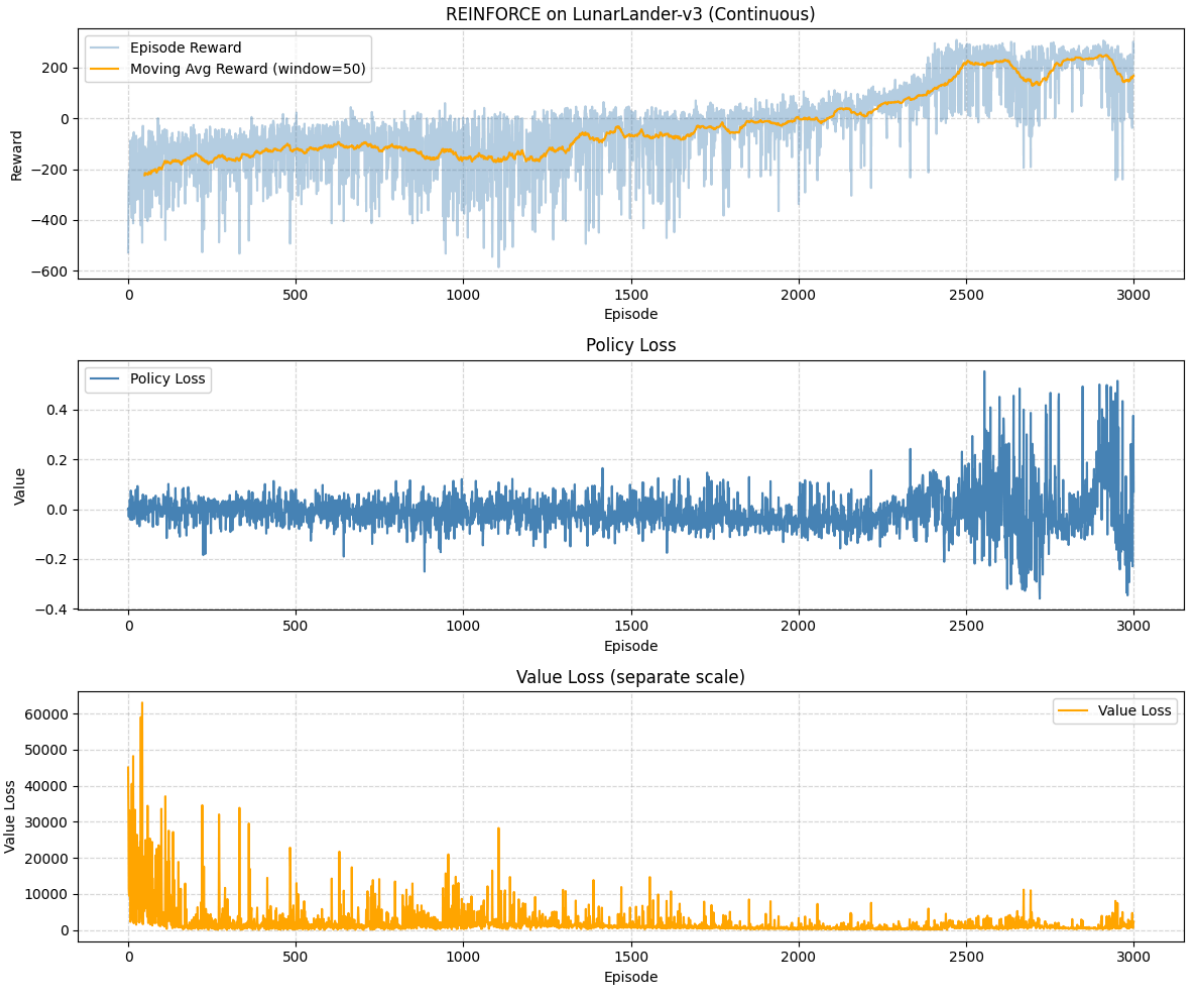

Figure 1: Training performance: episode reward with moving average (top), policy loss (middle), and value loss (bottom).

In the early episodes, rewards are highly variable and frequently negative, with individual episode rewards falling as low as -600. The moving average remains negative until around

episode 1000, after which it rises gradually. By episodes 2500–3000, it stabilizes near 200, satisfying the training objective.

The policy loss fluctuates around zero throughout training, with no clear downward trend. This is expected, as the policy loss does not indicate convergence in the same way as supervised learning losses.

The value loss starts extremely large in the early episodes (exceeding 60000) and drops rapidly within the first 200–500 episodes, indicating that the value network quickly learns a rough estimate of the return. It continues to decrease gradually and stabilizes near zero after episode 1000.

4.2. Best Episode Video

The best episode videos recorded during evaluation can be found in the results folder.

5. Conclusion

This report presented an implementation of REINFORCE with baseline applied to the LunarLander-v3 task. The four core components specified in the assignment were implemented: an actor-critic policy consisting of a policy network and a value network, stochastic action sampling from a Gaussian distribution during training with deterministic actions during evaluation, trajectory collection over a full episode before each network update, and return and advantage estimation using Monte Carlo returns with baseline subtraction and normalization. The trained agent achieved an average episode reward near 200, demonstrating reliable landing between the flags.

6. References

Achiam, J. (2018). *Spinning Up in Deep Reinforcement Learning*. OpenAI. https://spinningup.openai.com

7. Acknowledgments

This report was written with the assistance of Claude (Anthropic, 2025), a large language model, for grammar checking, phrasing, and structuring.